

Natural Language Processing

Fabian Bosshard

February 4, 2026

Contents

1	Useful formulae	2
2	Text Classification	3
2.1	Text preprocessing	3
2.2	Bag of words	3
2.2.1	Heaps' law	3
2.2.2	Zipf's law	4
2.3	Evaluating text classifiers	4
2.3.1	Evaluating a Binary Text Classifier	4
2.3.2	Evaluating multi-class classifiers	4
3	Text search and clustering	5
3.1	tf-idf: term frequency - inverse document frequency	5
3.1.1	Scoring documents	5
3.2	Evaluating search engines	5
4	Language models and word embeddings	6
4.1	Markov model	6
4.2	Evaluating	6
4.2.1	Perplexity	6
4.3	Embeddings	6
4.3.1	WordNet	7
4.3.2	Distributional hypothesis of Harris (1968)	7
4.3.3	Sparse versus dense vectors	7
4.4	Word2vec	7
4.5	Properties of Word Embeddings	8
5	Sequence classification and labelling	9
5.1	Named entity recognition (NER)	9
6	Sequence-to-sequence model	10

1 Useful formulae

Conditional probability: Let $(\Omega, \mathcal{A}, P)$ be a probability space. Recall that the conditional probability of an $A \in \mathcal{A}$ given $B \in \mathcal{A}$ is defined as

$$P(A | B) := \begin{cases} \frac{P(A \cap B)}{P(B)} & P(B) > 0 \\ 0 & P(B) = 0 \end{cases} \quad (1.1)$$

Theorem 1.1 (Chain rule of probability). For any events $A_1, \dots, A_n$:

$$P\left(\bigcap_{i=1}^n A_i\right) = \prod_{k=1}^n P\left(A_k \mid \bigcap_{j=1}^{k-1} A_j\right) \quad (1.2)$$

e.g. with 4 variables: $P(A, B, C, D) = P(A) P(B | A) P(C | A, B) P(D | A, B, C)$. $\triangleleft$

Proof. Apply Equation 1.1 repeatedly. $\square$

Cosine similarity between two vectors $\mathbf{A}$ and $\mathbf{B}$:

$$\text{similarity} = \cos(\angle(\mathbf{A}, \mathbf{B})) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (1.3)$$

2 Text Classification

2.1 Text preprocessing

Prior to tokenization:

- remove markup (non-content information), e.g. <HTML> tags
- lowercase text to reduce vocabulary size (can lose information: e.g. "WHO" vs "who")
- remove punctuation

After tokenization:

- remove stopwords¹ (convey little information about the topic of a text)
- remove low-frequency words (since insufficient information present to determine correlation between word presence and class)

less common activities:

- stemming or lemmatization to reduce vocabulary size
- spelling correction

ASCII encoding keyboard $\rightarrow [1, 128]$

UTF-8 encoding very vast (160 languages, 149k Unicode characters)

Definition 2.1 (Morpheme). Smallest linguistic unit that has semantic meaning. ◀

2.2 Bag of words

Vocabulary of document provides most important signal; # occurrences of words provides further information.

Bag-of-words model:

- represent documents as vectors of word counts
- sparse representation (long vector with many zeros)
- completely ignores word order
- extension to include n -grams can increase performance: but greatly increases number of dimensions, so more data is then needed

usually have far fewer documents than vocabulary terms, which means fewer examples than features

- means multiple settings of parameters can explain observed class labels
- strong regularization needed to prevent overfitting

2.2.1 Heaps' law

Vocabulary grows with approximately the square root of document/collection length:

$$V(l) \propto l^\beta, \quad \beta \approx 0.5$$

¹high-frequency terms in language, e.g. "the", "is", "and"

2.2.2 Zipf's law

Define as the collection term frequency ctf_t of a token t ("token frequency") as the number of times it occurs in the entire corpus:

$$\text{ctf}_t := \sum_{d \in \mathcal{D}} \text{count}(t, d)$$

Token frequency approximately proportional to the inverse of its rank:

$$\text{ctf}_t \propto \frac{1}{\text{rank}(t)^s}, \quad s \approx 1$$

Most common word occurs approximately twice as often as the next common one, three times as often as the third most common, and so on.

2.3 Evaluating text classifiers

2.3.1 Evaluating a Binary Text Classifier

First thing to investigate is the confusion matrix:

		PREDICTED CLASS	
		Pos	Neg
TRUE CLASS	Pos	TP: true positives	FN: false negatives
	Neg	FP: false positives	TN: true negatives

- Accuracy = % of correct predictions:

$$\text{Accuracy} = \frac{\text{TN} + \text{TP}}{\text{TN} + \text{TP} + \text{FN} + \text{FP}}$$

- Precision = % of positive predictions that were correct:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- Recall = % of positive instances that were found by model:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- F-measure = harmonic mean of precision and recall:

$$F_1 = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}}$$

- AUC = area under the ROC curve is often used as a single measure that doesn't depend on the confidence threshold used to make predictions with the model

2.3.2 Evaluating multi-class classifiers

if there are n classes,

- confusion matrix will be $n \times n$
- by considering it positive class in a one-vs-all setting
- combine each class's precision and recall values into a single measure:
 - **macro-average**: average over classes weighting each class the same
 - **micro-average**: average over classes weighting each class by the number of data-points in it

3 Text search and clustering

3.1 tf-idf: term frequency - inverse document frequency

Definition 3.1 (inverse document frequency).

$$\text{idf}_t = \log \frac{N}{\text{df}_t} \quad (3.1)$$

where df_t is the number of documents in the corpus which contains t (document frequency), while N is the number of documents in total (corpus size). ◀

high $\text{idf} \Rightarrow$ few documents matching; $\text{idf}_t = 0 \Rightarrow t$ is in every document ($\log 1 = 0$).

Definition 3.2 (term frequency).

$$\text{tf}_{t,d} = \begin{cases} 1 + \log_{10}(\text{count}(t, d)) & \text{count}(t, d) > 0 \\ 0 & \text{otherwise} \end{cases} \quad (3.2)$$

a term t which occur 0 times in a document d has $\text{tf}_{t,d} = 0$; if it occurs once, $\text{tf}_{t,d} = 1 + \log_{10}(1) = 1$; if it occurs 10 times, $\text{tf}_{t,d} = 1 + \log_{10}(10) = 2$; and so on. ◀

Definition 3.3 (tf-idf).

$$\text{tf-idf}(t, d) = \text{tf}_{t,d} \cdot \text{idf}_t \quad (3.3)$$

Definition 3.4 (BM25). goto method for term-based text retrieval...

$$\sum_{t \in q} \text{idf}_t \frac{\text{tf}_{t,d}}{\text{tf}_{t,d} + k \left(1 - b + b \frac{|d|}{d_{\text{avg}}} \right)}$$

- t terms in q query
- $k \rightarrow$ adjusts balance between term frequency and IDF
- $b \rightarrow$ controls the importance of document length normalization
- $|d|$ is length of the document; d_{avg} is the average on the document collection ◀

3.1.1 Scoring documents

Vector space model treats tf-idf values for all terms in document as a vector representation:

$$\mathbf{d} = [\text{tf-idf}(t_1, d), \text{tf-idf}(t_2, d), \dots, \text{tf-idf}(t_n, d)]$$

similarity between query vector $\mathbf{q}$ and document vector $\mathbf{d}$ computed using cosine similarity:

$$\text{score}(q, d) = \cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q}}{\|\mathbf{q}\|} \cdot \frac{\mathbf{d}}{\|\mathbf{d}\|}$$

3.2 Evaluating search engines

- precision at depth k : $P@k = (\# \text{ relevant docs in top } k)/k$
- recall at depth k : $R@k = (\# \text{ relevant docs in top } k)/(\# \text{ relevant docs in total})$
- F-measure at depth k :

$$F_1@k = \frac{1}{((1/P@k + 1/R@k)/2)}$$

- AveP = MAP: mean average precision (estimate area under precision/recall curve)

$$\text{AveP} = \frac{\sum_{k=1}^n P@k \text{ rel}(k)}{\# \text{ relevant docs}}$$

- $\text{bool rel}(k) = 1$ if document at position k is relevant

4 Language models and word embeddings

4.1 Markov model

$$P(\text{croquet} \mid \text{play}) = \frac{N(\text{play croquet})}{N(\text{play})}, \quad P(\text{croquet} \mid \text{to play}) = \frac{N(\text{to play croquet})}{N(\text{to play})}$$

Longer n -grams give better prediction but also less chance of finding the exact same sequence.

Smoothing: add a small constant to all counts before estimating probabilities, e.g. Laplace (with bigram assumption):

$$P_{\text{Laplace}}(w_n \mid w_{n-1}) = \frac{\text{count}(w_{n-1} w_n) + 1}{\sum_w (\text{count}(w_{n-1} w) + 1)} = \frac{\text{count}(w_{n-1} w_n) + 1}{\text{count}(w_{n-1}) + V}$$

Backoff: if an n -gram is unknown, use $(n - 1)$ -gram

Interpolate between n -grams, e.g. trigram-bigram-unigram:

$$P(w_n \mid w_{n-1} w_{n-2}) = \lambda_1 P(w_n \mid w_{n-1} w_{n-2}) + \lambda_2 P(w_n \mid w_{n-1}) + \lambda_3 P(w_n), \quad \sum_i \lambda_i = 1$$

choose lambdas that maximize probability on held-out **development** data

4.2 Evaluating

extrinsic: use model in a downstream task (e.g. as a spelling corrector) and evaluate performance on that task

intrinsic:

- train parameters of model on training set and test model performance on held-out dataset
- use likelihood that model would produce the new data to evaluate how well it is doing

4.2.1 Perplexity

of language model quantifies level of surprise/confusion at seeing new text

- measures how unlikely the observed data is under the model

Calculating perplexity:

1. **compute probability** of observed sequence $P(w_1, w_2, \dots, w_n)$ under model
 - e.g. for a bigram m: $P(w_1, \dots, w_n) = P(w_1) P(w_2 \mid w_1) P(w_3 \mid w_2) \cdots P(w_n \mid w_{n-1})$
2. **normalize probability** for length of text sequence
 - i.e. compute “average” per-word probability: $\sqrt[n]{P(w_1, w_2, \dots, w_n)}$
3. **invert probability** to compute uncertainty:

$$\text{Perplexity}(w) = (1/P(w_1, w_2, \dots, w_n))^{1/n} = \sqrt[n]{\frac{1}{P(w_1, w_2, \dots, w_n)}}$$

4.3 Embeddings

To overcome the problem of data sparsity in n -gram language models it would be good to have a similar representation for similar words.

4.3.1 WordNet

WordNet organizes lexical information in terms of word meanings (senses), rather than word forms (actual words). In that respect, WordNet resembles a thesaurus more than a dictionary.

- attempt to model human lexical memory
- is essentially an ontology (a carving up of the world's meanings)
- no longer SOTA, since difficult to build, language-specific, incomplete, still difficult to quantify similarity

4.3.2 Distributional hypothesis of Harris (1968)

words that occur in the same contexts tend to have similar meanings.

- **Practical implication:**
We can find similar words by comparing their distribution over contexts.
- Count how often words appear in several contexts; contexts become dimensions and target words w, v become vectors $\mathbf{w}, \mathbf{v}$, then compute $\cos(\mathbf{w}, \mathbf{v})$. gives a value in $[0, 1]$ of similarity.
- Then apply dimensionality reduction (SVD / Latent Semantic Analysis): reducing millions of contexts to hundreds of dimensions, while keeping as much of the info as possible (now it's **dense** embedding).

While tf-idf vectors are very long and full of 0s (sparse), learnt vectors are shorter and with full of non-0 (dense).

4.3.3 Sparse versus dense vectors

Count-based (or tf-idf) vectors are

- **long** (length $|V| = 20,000$ to $50,000$)
- **sparse** (most elements are zero)

Alternative: learn vectors which are

- **short** (length = 50 to 1000)
- **dense** (most elements are non-zero)

Why dense vectors?

- short vectors may be easier to use as *features* in machine learning (fewer weights to learn)
- may *generalize* better than explicit counts
- empirically, work better

Vocabulary, all the words
your model covers

4.4 Word2vec

Train a classifier on a binary prediction task: "Is word w likely to show up in context c ?"

We don't care about the task, but we take learned classifier weights as word embeddings.

big idea: **self-supervision**

Different versions, we focus on **skip-gram** with negative sampling (SGNS).

Skip-gram: given the central word, predict context.

One context word c :

$$P(+ | w, c) = \sigma(\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{c} \cdot \mathbf{w})}$$

We have lots of context words. We'll assume independence and just multiply them:

$$P(+ | w, c_{1:L}) = \prod_{i=1}^L \sigma(\mathbf{c}_i \cdot \mathbf{w})$$

$$\log P(+ | w, c_{1:L}) = \sum_{i=1}^L \log \sigma(\mathbf{c}_i \cdot \mathbf{w})$$

Train via SGD!

Word weights will be adjusted such that

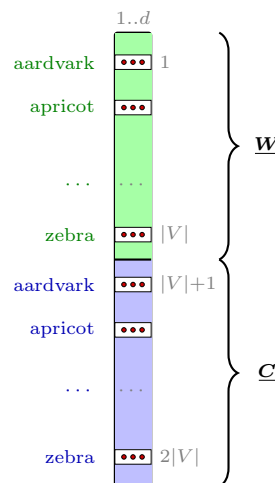
- positive pairs will be more likely
- negative pairs will be less likely

over the entire training set.

SGNS learns 2 sets of embeddings:

- target embeddings matrix $\underline{\mathbf{W}}$
- context embeddings matrix $\underline{\mathbf{C}}$

it is common to just add them together, representing word i as $\mathbf{w}_i + \mathbf{c}_i$.



Fasttext

- addresses problem of unknown words
- imagine language with a lot of morphology
- uses subword segmentation to deal with this

4.5 Properties of Word Embeddings

many interesting and somewhat surprising properties

semantic clustering:

- neighbors in embedding space are semantically related to one another

provide dense distributed representation

- where individual dimensions aren't usually interpretable

but translation in the space is meaningful

- certain directions in the space have meaning
- which means that semantics is often additive
- and analogies are encoded in the embedding: e.g. **man** is to **woman** as **king** is to **queen**

5 Sequence classification and labelling

CRF: conditional random fields.

Part of speech (POS): assigning to each word a word class (noun, verb etc), so a map from words to classes.

5.1 Named entity recognition (NER)

Similar to POS but instead of noun etc classes like “person”, “organisation”, etc.

NER can be hard because of:

- *segmentation*: in POS tagging each word gets one tag, while in NER entities can be phrases
- *type ambiguity*: same word/phrase can have many types depending on context

6 Sequence-to-sequence model

Soft-Attention produces weighted average over input embeddings:

$$w_{i,j} = P(j | i) = \text{softmax}(\text{similarity}(\mathbf{h}_{i-1}, \mathbf{e}_j)), \quad \mathbf{z}_i = \sum_j w_{i,j} \mathbf{e}_j$$

Calculating similarity:

- **additive** attention: concatenate decoder state and encoder embedding in a feedforward network:

$$\text{similarity}(\mathbf{h}_{i-1}, \mathbf{e}_j) = \text{FFNN}(\mathbf{h}_{i-1} \parallel \mathbf{e}_j)$$

- **multiplicative** attention: dot product of encoder vector and decoder previous state,

$$\text{similarity}(\mathbf{h}_{i-1}, \mathbf{e}_j) = \frac{\mathbf{h}_{i-1} \cdot \mathbf{e}_j}{\sqrt{d}}$$

where d is the embedding size.

- we divide by $\sqrt{d}$ so that the standard deviation remains 1:

$$\text{if } a_i, b_i \sim \mathcal{N}(0, 1) \Rightarrow \text{Var}[a_i b_i] = 1$$

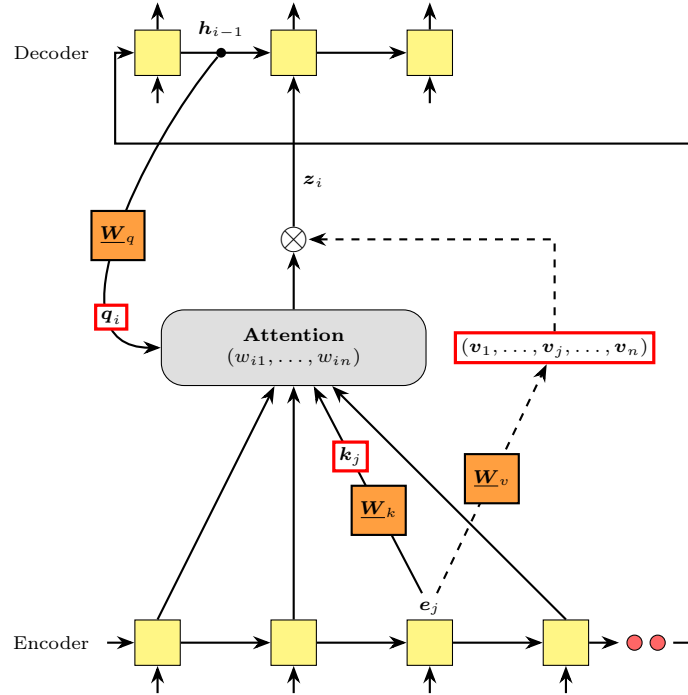
$$\Rightarrow \text{Var}[\mathbf{a} \cdot \mathbf{b}] = \sum_{i=1}^d \text{Var}[a_i b_i] = d$$

Generalizing the notation with queries (what is being looked up), keys (index of what to find), values (stored information at key):

$$\mathbf{z}_i = \sum_j \text{softmax}\left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}}\right) \mathbf{v}_j$$

where the queries, keys, and values are usually obtained by a linear transformation:

$$\mathbf{q}_i = \mathbf{W}_q \mathbf{h}_{i-1}, \quad \mathbf{k}_i = \mathbf{W}_k \mathbf{e}_j, \quad \mathbf{v}_j = \mathbf{W}_v \mathbf{e}_j, \quad \mathbf{W}_{q,k,v} \in \mathbb{R}^{d \times d}$$



Interpreting queries, keys, values with a hypothetical example:

- **query**: what is being looked up
need adjective that describes a person
- **key**: index of what to find
have adjective that describes people & animals
- **value**: stored information at key
word is “friendly”, which in Italian could be translated to “simpatico”